CUDA Kernel Optimization: Mathematical Analysis and GPU Execution Logic

Tim Li

October 19, 2026

1 Introduction

This project implements and optimizes four basic matrix operations in CUDA: transpose, trace, addition, and multiplication. Each one stresses the GPU in a different way. Transpose rearranges data, trace is a parallel reduction, addition works element by element, and multiplication repeats multiply-accumulate operations.

Our main goal is to see how each operation maps onto GPU threads and blocks, and how optimizations affect performance, memory use, and resource efficiency. For each operation, we start with a basic kernel and then try several optimizations: thread configuration, memory access patterns, shared memory, and loop organization.

These optimizations involve a trade-off between energy, storage, and speed. Global memory is large, but each access costs more time and energy than registers or shared memory. Reusing data to reduce global memory traffic can therefore improve both speed and energy efficiency. On-chip storage is limited, though. Using more shared memory or registers can reduce the number of resident blocks and warps when resource limits are reached, potentially lowering occupancy. This means the kernel with the least memory traffic is not always the fastest.

How these factors play out differs for each operation. We examine this through mathematical analysis, the CUDA execution model, and performance experiments. Without power measurements, we will not infer energy use from execution time alone.

2 Common CUDA Execution Model

CUDA organizes threads into a hierarchy. A grid contains blocks, and each block contains a number of threads chosen by the programmer. The global index of a thread in a one-dimensional launch is

$$k = \text{blockIdx.x} \times \text{blockDim.x} + \text{threadIdx.x}. \tag{1}$$

The GPU schedules threads in groups of 32 called warps. Threads in the same block can share data through shared memory and synchronize with barriers; the kernels that need one use `__syncthreads()`. Different blocks cannot synchronize with each other inside a single kernel launch. When one stage of the trace reduction depends on the results of another, we therefore use separate kernel launches in the same stream, which run in order.

CUDA provides three main memory spaces:

- **Registers:** private to each thread and the fastest to access.
- **Shared Memory:** on-chip memory shared by the threads in one block.

- **Global Memory:** accessible to all threads, but with the highest latency.

For a row-major matrix $A \in \mathbb{R}^{M \times N}$, the element at row i and column j is stored at the linear offset

$$\text{offset}(i, j) = iN + j. \tag{2}$$

In a two-dimensional kernel, each thread is mapped to a matrix element by:

$$\text{col} = \text{blockIdx.x} \times \text{blockDim.x} + \text{threadIdx.x}, \tag{3}$$

$$\text{row} = \text{blockIdx.y} \times \text{blockDim.y} + \text{threadIdx.y}. \tag{4}$$

Boundary checks prevent invalid global-memory accesses. In tiled kernels, threads outside the valid output region still participate in tile loading and block-wide barriers.

Two ideas matter for performance throughout this report. The first is memory coalescing. When neighboring threads access neighboring addresses, the hardware combines their requests into fewer memory transactions. Shared-memory tiling can reduce repeated global reads, or reorder accesses so they coalesce.

The second is occupancy, the ratio of active warps on a streaming multiprocessor to the maximum it supports. Occupancy is limited by registers per thread, shared memory per block, threads per block, and the hardware limits of the GPU. Higher occupancy does not always mean faster code. A good kernel balances memory traffic, instruction count, data reuse, and resource use.

3 Matrix Transpose

3.1 Mathematical Model and CUDA Design

For an $N \times N$ input matrix A, the output matrix B satisfies

$$B_{j,i} = A_{i,j}, \qquad 0 \le i, j < N.$$

Both matrices are stored in row-major order in separate buffers. Each thread copies one element. A two-dimensional grid of two-dimensional blocks covers the matrix, and boundary checks handle sizes that are not multiples of the block dimensions.

3.2 Naive Implementation

Source code: Naive transpose implementation.

The program takes N and the block dimensions (b_x, b_y) as inputs. The host allocates device buffers, copies the FP32 input to the GPU, and launches `transposeNaive`. Each thread computes its row and column from its block and thread indices, then copies one value directly to its transposed position in global memory. The threads are independent, so the kernel needs no barrier.

For timing, we run 10 warm-up launches and then use CUDA events to time 100 launches, reporting the average. Allocation, host–device transfers, and the correctness check are excluded from the timing. After the run, the output is copied back to the host and compared element by element with the expected result.

3.3 Block-Shape Sweep

In an initial sweep at $N = 4096$, every configuration used 256 threads per block (Table 1). These measurements come from an earlier session, so we keep them separate from the comparison later in this section.

Table 1: Initial naive-transpose block-shape sweep ($N = 4096$).

Block dimensions	Threads/block	Approximate time (ms)
16×16	256	5.15
32×8	256	10.9
8×32	256	2.7
4×64	256	2.8

In the naive kernel, neighboring threads in the x direction read neighboring input elements but write to addresses N elements apart. The timing differences fit a change in the warp-level memory access pattern, but profiling is needed to confirm the actual bottleneck. We consider three optimizations:

1. **Block-shape tuning:** change how the threads of a warp are laid out while keeping the number of threads per block fixed.
2. **Shared-memory tiling:** stage data in shared memory so that both global reads and global writes are contiguous.
3. **Shared-memory padding:** add one extra column to the tile to reduce bank conflicts when the tile is read in transposed order.

3.4 Shared-Memory Tiled Implementation

Source code: Tiled transpose implementation.

This version fixes both the block and the tile at 16×16. Each thread first loads one input value into `tile[ty][tx]` in shared memory. A `__syncthreads()` call, executed by every thread, makes sure the whole tile is loaded. The block coordinates are then swapped, and each thread reads `tile[tx][ty]` so that neighboring threads write neighboring addresses in the output. As a result, the thread that writes an element is generally not the thread that loaded it. Separate bounds checks on the load and the store keep the kernel correct at the matrix edges. Initialization, validation, and timing are the same as for the naive version.

3.5 Shared-Memory Padding

Source code: Padded transpose implementation.

The shared array changes from `tile[TILE][TILE]` to `tile[TILE][TILE+1]`. With `TILE=16`, the logical tile is still 16×16, but the allocation becomes 16×17. The extra column is never used. Its only effect is to change the row stride from 64 to 68 bytes, which changes how elements map to shared-memory banks when threads read `tile[tx][ty]`.

Everything else is unchanged: block dimensions, indexing, boundary checks, and the single barrier. Each block still has 256 threads (8 warps). Shared memory per block grows from 1024 to 1088 bytes, an increase of 64 bytes (6.25%). The goal is to reduce shared-memory bank conflicts without changing the global-memory accesses.

3.6 Timing Results

We tested all three implementations on an NVIDIA Orin with FP32 data and 16×16 blocks. Each program was run three times, and Table 2 reports the median of the three average kernel times. Every run passed the correctness check with zero mismatches, including the non-divisible size $N = 1000$.

Table 2: Transpose kernel times with 16×16 blocks on NVIDIA Orin.

N	Naive (ms)	Tiled (ms)	Padded (ms)	Tiled speedup	Padded speedup
1000	0.295475	0.163317	0.152165	1.81×	1.94×
4096	4.216940	2.459742	2.277422	1.71×	1.85×

Here the speedup is the naive time divided by the optimized time:

$$\text{Speedup} = \frac{T_{\text{naive}}}{T_{\text{optimized}}}.$$

Tiling reduced kernel time by 44.7% at $N = 1000$ and 41.7% at $N = 4096$. It costs 1 KiB of shared memory per block and one barrier per block, but these costs were small compared with the gain. Because tiling does not change the number of logical reads or writes, the gain is consistent with better-organized global memory accesses.

Padding gave a further 6.8% reduction at $N = 1000$ and 7.4% at $N = 4096$ relative to the unpadded tiled kernel. It needs only 64 extra bytes of shared memory per block and no extra threads or barriers. The padded version was the fastest at both sizes. These results concern kernel execution time only; memory allocation and host–device transfers are excluded.

3.7 Profiling Evidence

Timing alone does not show why the kernels differ, so we used Nsight Compute to profile one launch of each implementation at $N = 4096$, after 10 warm-up launches. All versions used 16×16 blocks and a 256×256 grid, which gives 256 threads (8 warps) per block and 65,536 blocks in total. Table 3 lists the selected metrics.

Table 3: Profiler comparison of the three transpose implementations ($N = 4096$).

Metric	Naive	Tiled	Padded
Global-store L1/TEX sectors	8,388,608	2,097,152	2,097,152
Useful bytes/store sector	8	32	32
Shared-load bank conflicts	—	3,800,085	582,760
Shared-store bank conflicts	—	50,548	547,426
Registers/thread	16	16	16
User static shared memory/block (B)	0	1024	1088
Achieved occupancy (%)	80.18	88.87	88.81

The naive kernel does not use shared memory, so its bank-conflict entries are marked with dashes. Static shared memory excludes driver overhead.

Global-memory access. All three versions issued the same number of global-store requests (524,288) and the same number of global-load sectors (2,097,152). Tiling cut the number of L1/TEX store sectors by 75%, and the useful bytes per 32-byte store sector rose from 8 to 32. So tiling mainly improved the write pattern. These are L1/TEX metrics and do not imply an equal drop in DRAM traffic, which we could not measure.

Padding trade-off. Padding reduced shared-load bank conflicts by 84.7%, but it also raised shared-store conflicts from 50,548 to 547,426. It improved the load pattern without removing every conflict. Register use was unchanged, and achieved occupancy was almost identical for the tiled and padded kernels, so the extra speedup is better explained by the shared-memory access pattern than by higher occupancy.

3.8 Summary

Compared with the naive kernel, the padded version was 1.94× faster at $N = 1000$ and 1.85× faster at $N = 4096$, and all correctness checks passed. Tiling gave most of the improvement. Padding added a smaller gain for 64 extra bytes of shared memory per block, so a small layout change can be worthwhile even when it trades fewer load conflicts for more store conflicts. We left out program timings taken under the profiler, because replay and data collection add overhead.

4 Matrix Trace

4.1 Mathematical Model and CUDA Design

For an $N \times N$ matrix A, the trace is the scalar

$$\operatorname{tr}(A) = \sum_{i=0}^{N-1} A_{i,i}.$$

The input is a full FP32 matrix in row-major storage. Only the N diagonal elements contribute to the result, and $A_{i,i}$ is located at `matrix[i*(N+1)]`.

The GPU uses a hierarchical reduction. Each thread loads a diagonal element, each block reduces its elements to one partial sum, and later kernels reduce the contiguous array of partial sums to a single scalar. Threads whose input index is out of range contribute zero, so every thread still reaches the block-wide barriers. The first stage reads with stride $N + 1$, while later stages read with stride 1.

4.2 Naive Implementation

Source code: Naive trace implementation.

The program takes N and a power-of-two block size B. The kernel receives an input pointer, an output pointer for the partial sums, the number of inputs, and the access stride. The host allocates the matrix and two partial-sum buffers, launches all stages in the same stream, and copies the final scalar back to the CPU for validation.

In the baseline, each thread loads one value into dynamic shared memory. The reduction is interleaved: the stride starts at 1 and doubles each round, and only threads with `tid % (2*s) == 0` perform an addition. Thread 0 writes the block result. The first stage uses $\lceil N/B \rceil$ blocks, and each block allocates $4B$ bytes of shared memory.

4.3 Measurement Procedure

All timing experiments ran on an NVIDIA Orin with FP32 accumulation on the GPU. A *trace* is one complete reduction. Every configuration reported here needs two kernel launches per trace, so the 10 warm-up traces comprise 20 launches. Each run then times 100 traces with CUDA events that cover all reduction stages and the gaps between launches. Allocation, initialization, transfers, and CPU validation are excluded. The tables report the median of three runs, where each run gives the average time per trace. Each experiment below was a separate round, and we report the rounds separately rather than mixing them.

4.4 Sequential-Addressing Reduction

Source code: Sequential-addressing trace implementation.

The interleaved baseline activates scattered threads in each round. The sequential version follows the organization described by Harris [1]. It starts at stride $B/2$, halves the stride each round, and lets only the threads with `tid < s` add. For $B = 256$, the active ranges are 0–127, 0–63, 0–31, and so on. This replaces the modulo test with a threshold test and keeps the active threads together at the start of the block.

Once $s < 32$, only part of the first warp does useful additions. This describes which lanes do arithmetic, not necessarily divergent branches, because the compiler may implement short conditionals with predicated instructions [2]. All threads still execute the block-wide barriers.

To isolate the reduction organization, the first comparison fixed $B = 256$. Both versions used the same inputs, stage counts, global-memory indexing, and shared-memory allocation. Each block ran eight reduction rounds and nine barriers, including the one after loading, and we added no manual loop unrolling. Table 4 gives the median complete-trace times of three runs, measured as described above.

Table 4: Naive versus sequential reduction with 256 threads per block.

N	Naive (ms)	Sequential (ms)	Speedup	Time reduction
1000	0.012584	0.010312	1.22×	18.1%
4096	0.014928	0.012377	1.21×	17.1%

Sequential addressing was faster at both sizes, but timing alone does not show why. We therefore turn to profiling (Section 4.9), which used 128 threads per block rather than the 256 used here. At that setting, the sequential version executed fewer instructions and shared-memory wavefronts. All three profiled variants reported 100% branch efficiency and no divergent branches. The results therefore support less execution work as the explanation, not reduced branch divergence. We interpret the metrics following the Nsight Compute guide [3].

4.5 Block-Size Comparison

A separate experiment fixed $N = 4096$ and tested $B = 128$, 256, and 512 for both implementations (Table 5). A block contains $B/32$ warps, and the second stage uses a single block in every case.

Both implementations were fastest at $B = 128$. Smaller blocks need fewer reduction rounds per block and produce more first-stage blocks, but these effects change together, so the data cannot say which one matters most or whether smaller blocks are always better. We used $B = 128$ for the remaining experiments.

Table 5: Block-size comparison at $N = 4096$, including all stages.

B	First-stage blocks	Shared/block (B)	Naive (ms)	Sequential (ms)
128	32	512	0.014097	0.011946
256	16	1024	0.014923	0.012394
512	8	2048	0.016459	0.012878

4.6 Pair-Load Implementation

Source code: Pair-load trace implementation.

Starting from the sequential version, each thread reads up to two inputs and adds them in a register before writing one value to shared memory. For block index b and thread index t, the input indices are

$$i = 2Bb + t, \qquad j = i + B.$$

Each load has its own bounds check. In the first stage these are diagonal indices, and in later stages they index the partial-sum array. The first stage then needs $\lceil N/(2B) \rceil$ blocks. This changes how work is divided among threads, so it is more than loop unrolling.

At $B = 128$, the first stage drops from 8 to 4 blocks for $N = 1000$ and from 32 to 16 blocks for $N = 4096$. Both versions still use two kernels, 512 bytes of shared memory per block, and seven reduction rounds with eight barriers per block. Adding two values per thread reduces the number of partial sums, but it also leaves fewer blocks to run in parallel.

4.7 Pair-Load Timing Results

We compared the sequential and pair-load versions again at $B = 128$ in a fresh round (Table 6). A positive time change means pair-load was slower.

Table 6: Sequential versus pair-load at 128 threads per block.

N	Sequential (ms)	Pair-load (ms)	Pair-load time change
1000	0.009925	0.010000	+0.76%
4096	0.011975	0.012013	+0.32%

The differences are only 0.075 and 0.038 microseconds per trace, and the three-run ranges overlap at both sizes. Pair-load therefore shows no clear speedup, and the data do not show a significant slowdown either. The kernel count and the barriers per block are unchanged, while the first-stage block count is halved. A likely explanation is that the saved work is offset by lower parallelism, which we check in Section 4.9.

4.8 Numerical Accuracy

The CPU reference sums the FP32 diagonal values in FP64. A run passes if

$$|T_{\text{GPU}} - T_{\text{ref}}| \le 10^{-5} + 10^{-5}|T_{\text{ref}}|.$$

All checks passed, and the largest relative error was about 6.68×10^{-8}. Table 7 lists the absolute errors.

Table 7: Absolute error relative to the CPU FP64 reference. Column labels give the implementation and threads per block.

N	Naive, 256	Seq., 256	Seq., 128	Pair, 128
1000	3.3225×10^{-5}	2.7810×10^{-5}	2.7076×10^{-6}	2.7810×10^{-5}
4096	5.0709×10^{-5}	5.0709×10^{-5}	5.0709×10^{-5}	5.0709×10^{-5}

At $N = 1000$ the error depended on the summation order, from 2.7×10^{-6} to 3.3×10^{-5}, while at $N = 4096$ all four configurations gave the same error. Changing the reduction tree changes the rounding, so every version stays within tolerance but no summation order is shown to be generally more accurate. This is an accuracy comparison of FP32 implementations against an FP64 reference, not a performance comparison of FP32 and FP64.

4.9 Profiling Analysis

We used Nsight Compute to profile both reduction stages at $N = 4096$ with 128 threads per block. We skipped 20 warm-up launches and collected the next two, which make up one complete trace. All versions used 16 registers per thread and 512 bytes of dynamic shared memory per block. Table 8 summarizes the first stage.

Table 8: First-stage trace profiling at $N = 4096$, $B = 128$.

Metric	Naive	Sequential	Pair-load
Grid blocks	32	32	16
Achieved occupancy (%)	30.77	28.89	15.59
Executed warp instructions	29,216	12,832	7,232
Shared-memory wavefronts	2,368	928	464
Global-load L1/TEX sectors	4096	4096	4096
Useful bytes/global-load sector	4	4	4
Branch efficiency (%)	100	100	100

Reduction organization. Sequential addressing cut the first-stage executed warp instructions by 56.1% and the shared-memory wavefronts from 2,368 to 928. Global-load sectors, register use, and shared-memory allocation did not change. This supports less instruction and shared-memory work as the reason for the speedup. It does not support branch divergence or bank conflicts as the reason: all three versions reported 100% branch efficiency and no divergent branches, and the bank-conflict and excessive wavefront counters were zero in both stages. Note that 100% branch efficiency does not mean every lane does useful work, because the compiler can implement short conditionals with predication instead of divergent branches.

Diagonal access and available parallelism. Every first-stage version used only 4 useful bytes of each 32-byte load sector (12.5%), because the diagonal stride is the same in all of them. Pair-load halved the number of first-stage blocks and reduced instruction work, but achieved occupancy fell from 28.89% to 15.59%. The profiler reported about 0.33 waves per SM for sequential and 0.17 for pair-load, even though theoretical occupancy was 100% for both. So limited parallel work, not

register or shared-memory pressure, is the main cost of pair-load here. These L1/TEX counters do not measure DRAM traffic, and DRAM throughput was unavailable.

Final reduction stage. The second stage used one block in every version, so it ran on a single SM (Table 9). Pair-load cut the partial-sum input from 32 to 16 values but did not remove this kernel launch. It also executed more warp instructions than the sequential version (452 versus 401), and we did not isolate why. Only 4 to 6 stall samples were collected for this stage, which is too few to support a stall-based bottleneck claim.

Table 9: Second-stage trace profiling at $N = 4096$, $B = 128$.

Metric	Naive	Sequential	Pair-load
Grid blocks	1	1	1
Executed warp instructions	892	401	452
Global-load L1/TEX sectors	4	4	2
Achieved occupancy (%)	8.23	8.20	8.20

We used profiling to explain behavior, not to replace the timing measurements. Profiler durations were not summed or used as complete-trace times, and the contribution of launch overhead was not isolated.

4.10 Summary

With 256 threads per block, sequential addressing cut complete-trace time by 18.1% at $N = 1000$ and 17.1% at $N = 4096$. Profiling links this to fewer executed instructions and shared-memory wavefronts rather than to fewer divergent branches or bank conflicts. Among the tested block sizes, 128 threads per block was fastest at $N = 4096$. At that size, pair-load gave no clear further speedup: its median times were within 0.76% and 0.32% of sequential, with overlapping ranges. Pair-load did less work, but it also ran on fewer blocks and reached lower occupancy.

All three versions kept the inefficient diagonal loads and the single-block final stage. All correctness checks passed, with a largest relative error of about 6.68×10^{-8}. We keep sequential reduction with 128 threads per block for the tested workloads. The main lesson is that fewer instructions, blocks, or partial sums do not guarantee a shorter execution time, because available parallelism and the number of stages also matter.

5 Matrix Addition

5.1 Mathematical Model and CUDA Design

For two real $N \times N$ matrices A and B, matrix addition produces a matrix of the same size:

$$C_{i,j} = A_{i,j} + B_{i,j}, \qquad 0 \le i, j < N.$$

Each output requires one addition and is independent of all other outputs. The total arithmetic work is $O(N^2)$.

The implementation stores FP32 matrices in row-major order and treats them as arrays of $M = N^2$ elements. A one-dimensional grid assigns one output element to each thread. Neighboring threads access neighboring elements in both inputs and the output, providing a coalescing-friendly

access pattern. There are no cross-thread data dependencies, so no block-wide barriers are required. Each input is used once; shared-memory staging would introduce extra operations without providing cross-thread data reuse.

5.2 Native Implementation and Baseline Results

Source code: Native matrix-addition implementation.

The program accepts matrix dimension N and threads per block T. The host initializes and copies the inputs to the GPU, launches the kernel, and copies the output back for validation. The kernel receives two input pointers, one output pointer, and element count M. Thread t in block b uses index $i = bT + t$ and writes `C[i] = A[i] + B[i]` when $i < M$.

The grid contains $G = \lceil M/T \rceil$ blocks. The launched thread count is GT, and the launched warp count is $G\lceil T/32 \rceil$. These counts describe the full launch, not simultaneous residency or achieved occupancy. The kernel uses no explicit shared-memory allocation and no `__syncthreads()`.

All experiments ran on NVIDIA Orin with FP32 GPU arithmetic. Each run used 10 warm-up launches followed by 100 timed launches. CUDA events measured the launch batch, and elapsed time was divided by 100. Allocation, input initialization, transfers, and CPU validation were excluded. Tables report the median of three runs, each reporting an average time per addition. Separate experimental rounds are kept separate below.

Table 10: Initial native baseline with 256 threads per block.

N	Grid blocks	Time (ms)	Effective BW (GB/s)
1000	3907	0.140544	85.382
4096	65536	2.263596	88.941

Each output involves two FP32 reads and one FP32 write, giving 12 useful bytes per element. Effective bandwidth is computed as

$$BW_{\text{eff}} = \frac{12N^2}{t},$$

where t is time in seconds and bandwidth is in bytes per second. This is a useful-data rate, not measured DRAM bandwidth; repeated launches can be influenced by caching. The similar effective rates at both sizes are consistent with an important memory-processing cost, but timing alone does not identify the hardware bottleneck.

5.3 Optimization 1: Block-Size Adjustment

The first experiment changed T to 128, 256, and 512 while retaining the same kernel, inputs, and one-element-per-thread mapping. Table 11 includes the initial 256-thread measurements and the subsequent block-size tests. For these settings, each block contains $T/32$ warps.

Among the tested configurations, 128 threads per block gave the lowest median time at both sizes, reducing time relative to 256 threads by 2.05% and 2.00%, respectively. Increasing block size did not improve performance. At $N = 4096$, all configurations launched the same total number of threads and warps, but organized them into different numbers of blocks. Scheduling and resource residency are possible explanations for the difference; occupancy was not inferred from block size alone. The next experiment fixed $T = 128$.

Table 11: Native block-size comparison; median times from three runs.

N	T	Grid blocks	Warps/block	Time (ms)	BW (GB/s)
1000	128	7813	4	0.137666	87.168
1000	256	3907	8	0.140544	85.382
1000	512	1954	16	0.144888	82.823
4096	128	131072	4	2.218230	90.760
4096	256	65536	8	2.263596	88.941
4096	512	32768	16	2.338516	86.092

5.4 Optimization 2: Four Elements per Thread and Loop Unrolling

Source code: Unrolled matrix-addition implementation.

The second version assigns up to four output elements to each thread. For block b, thread t, and step k, the element index is

$$i_k = 4Tb + t + kT, \qquad k = 0, 1, 2, 3.$$

Each step checks $i_k < M$ independently. Neighboring threads still access consecutive elements at each step. The fixed four-step loop uses `#pragma unroll` to request compile-time expansion. The grid becomes $G_4 = \lceil M/(4T) \rceil$ blocks.

At $T = 128$, the block count decreases from 7813 to 1954 for $N = 1000$ and from 131072 to 32768 for $N = 4096$. The latter case reduces launched threads from 16,777,216 to 4,194,304 and launched warps from 524,288 to 131,072. Each output still requires the same one addition and 12 useful bytes. Both implementations launch one kernel per operation and require no block-wide barriers.

This experiment combines increased work per thread with loop expansion; it does not isolate loop unrolling alone. Less indexing or scheduling work per output may help, while fewer threads may reduce available parallelism. Expanded code may also affect register usage and code size. Register and occupancy changes require profiling rather than assumptions.

5.5 Unrolled Results and Accuracy

A fresh comparison alternated native and unrolled runs at $T = 128$. Table 12 uses this round's native measurements instead of the earlier block-size results.

Table 12: Native versus unrolled addition at 128 threads per block.

N	Native (ms)	Unrolled (ms)	Time reduction	Unrolled BW (GB/s)
1000	0.137806	0.132395	3.93%	90.638
4096	2.217070	2.108076	4.92%	95.503

All three unrolled runs were faster than all three native runs at each size in this comparison. The improvement cannot be explained by fewer kernel launches, fewer barriers, or less useful memory traffic, since these quantities were unchanged. Reduced execution overhead per output is a plausible explanation to examine with profiling.

The CPU FP64 reference adds the actual stored FP32 inputs. Every output is checked using

$$|C_i - C_i^{\text{ref}}| \le 10^{-6} + 10^{-5}|C_i^{\text{ref}}|.$$

All reported configurations passed with zero mismatches. The maximum absolute error was $5.96046448 \times 10^{-8}$ and the normalized infinity error was $2.98172308 \times 10^{-8}$, where

$$E_\infty = \frac{\max_i |C_i - C_i^{\mathrm{ref}}|}{\max_i |C_i^{\mathrm{ref}}|}.$$

This metric is distinct from the maximum elementwise relative error. The same errors were observed for native and unrolled versions, consistent with each output retaining the same single FP32 addition. Both versions also passed the irregular-size check at $N = 1001$. These tests assess FP32 accuracy; GPU FP64 performance was not tested.

5.6 Profiling Analysis

Nsight Compute profiled native and unrolled addition at $N = 4096$ with 128 threads per block. Ten warm-up launches were skipped, and the next kernel launch was collected. Table 13 summarizes the measurements. Profiling explains execution behavior; the ordinary CUDA-event measurements remain the basis for timing and speedup comparisons. Neither the profiler's kernel duration nor the program's average time under profiling replaces those results.

Table 13: Addition profiling at $N = 4096$, 128 threads per block.

Metric	Native	Unrolled
Grid blocks	131,072	32,768
Registers/thread	16	19
Theoretical occupancy (%)	100	100
Achieved occupancy (%)	86.90	96.01
Executed warp instructions	12,582,912	9,568,256
Global-load L1/TEX sectors	4,194,304	4,194,304
Global-store L1/TEX sectors	2,097,152	2,097,152
Useful bytes/load sector	32	32
Useful bytes/store sector	32	32
L2 throughput (%)	75.08	93.04
Compute (SM) throughput (%)	19.42	17.84
Branch efficiency (%)	100	100

Instruction work and register trade-off. The unrolled version executed 24.0% fewer warp instructions while computing the same number of outputs. This supports reduced instruction overhead as a contributor to the observed speedup. Register usage increased from 16 to 19 per thread, but theoretical occupancy remained 100% and achieved occupancy increased by 9.11 percentage points. Thus, the extra registers did not reduce the theoretical occupancy limit for this configuration. Although the grid was four times smaller, it still provided hundreds of waves per SM. The result shows that fewer launched threads need not imply lower achieved occupancy. Higher achieved occupancy is consistent with greater latency-hiding capacity, but does not by itself establish the cause of the speedup.

Memory behavior and remaining limitation. Both versions fully utilized the reported 32-byte global-load/store sectors and generated identical load and store sector counts. The unrolled

implementation therefore retained efficient access organization without reducing the measured L1/TEX sector traffic. Its L2 throughput increased from 75.08% to 93.04%, while compute throughput remained below 20%. In these reports, the summary memory-throughput value matched the L2 value. Together, these measurements support a remaining limitation in the memory path, particularly the reported L2 activity, rather than arithmetic throughput. DRAM throughput was unavailable, so 93.04% must not be interpreted as measured DRAM bandwidth utilization.

Long-scoreboard stalls dominated the sampled warp states in both versions, consistent with waiting for dependencies on L1/TEX memory operations. Their presence shows that memory dependency waits remain; sampled stall counts are not direct measurements of time saved. The roughly 24% instruction reduction produced only a 4.92% ordinary timing improvement at $N = 4096$, consistent with unchanged data movement remaining a substantial cost.

Divergence and synchronization. Both kernels reported 100% branch efficiency, zero divergent branches, and zero barrier-stall samples at the profiled size. Neither uses source-level shared-memory loads, stores, or block-wide barriers. Static and dynamic shared-memory allocations were both zero; the separately reported 1.02 kB per block was driver allocation. The measured improvement therefore cannot be attributed to removing divergence, shared-memory bank conflicts, or synchronization. These branch results apply to $N = 4096$; the irregular-size correctness test was not profiled.

5.7 Final Summary

Block-size tuning favored 128 threads per block among the tested settings, reducing median time by approximately 2% relative to 256 threads at both matrix sizes. A separate comparison at 128 threads per block showed that processing four elements per thread with loop expansion reduced time by another 3.93% at $N = 1000$ and 4.92% at $N = 4096$. These experiments combine a change in work distribution with loop expansion, so the latter gains are not attributed to loop unrolling alone.

Profiling showed fewer executed instructions, slightly higher register usage, and higher achieved occupancy without changing the global-memory sector counts. The unrolled implementation used the memory path more effectively, while memory dependency waits remained prominent. Both implementations passed all reported correctness checks, including $N = 1001$, with unchanged maximum absolute error of approximately 5.96×10^{-8}.

The four-element unrolled version with 128 threads per block is retained for the tested workloads. It provides a modest measured improvement with limited additional code and no observed accuracy penalty. GPU FP64 performance and the maximum supported matrix size were not established by these experiments.

6 Matrix Multiplication

6.1 Mathematical Model and CUDA Design

For two $N \times N$ matrices A and B, the output is

$$C_{i,j} = \sum_{k=0}^{N-1} A_{i,k} B_{k,j}.$$

Each output is a dot product of length N. The total work is $O(N^3)$, and we use the conventional count of $2N^3$ useful operations. All matrices are stored in row-major FP32. A two-dimensional grid maps each thread to one output element, and the thread accumulates it in a local FP32 scalar.

6.2 Naive Implementation

Source code: Naive matrix-multiplication implementation.

The program takes N, the two block dimensions, and an optional repetition count. The host allocates and initializes the two inputs and the output, copies the inputs to the GPU, launches the kernel, and copies the output back for validation. The kernel receives three matrix pointers and N. Each thread finds its row and column from its coordinates and loops over k to compute the full dot product. Threads outside the output range do nothing.

At a fixed k, neighboring x-direction threads in the same output row read the same element of A and adjacent elements of B. Each thread walks down a column of B with stride N, but this does not mean that neighboring threads make strided accesses. The naive kernel relies on hardware caching to reuse input data and uses no explicit shared memory or barriers.

6.3 Measurement Procedure

Timing runs on an NVIDIA Orin used 10 warm-up launches followed by 100 timed launches. CUDA events enclosed the 100-launch batch, and the elapsed time was divided by 100. Allocation, initialization, transfers, and CPU validation are excluded. The timing tables report the median of three independent runs, where each run gives the average time per multiplication. Effective compute throughput is $2N^3/(10^9 t)$ GFLOP/s, with t in seconds.

6.4 Optimization 1: Shared-Memory Tiling

Source code: Tiled matrix-multiplication implementation.

A square block of $T \times T$ threads computes one output tile. The computation runs in $\lceil N/T \rceil$ phases. In each phase, the threads cooperatively load one tile of A and one tile of B into shared memory, and each loaded value is then used for T different dot products in the block. The threads add T products to their running sums before the next phase.

Each phase has two block-wide barriers. The first makes the loaded tiles visible to all threads, and the second keeps the tiles from being overwritten while other threads are still reading them. Partial tiles at the matrix edge are filled with zeros. Every thread reaches both barriers, including threads outside the valid output region, so the kernel has no early return before the barriers. Bounds checks guard the individual loads and the final store.

The first comparison fixed the block and tile at 16×16, which keeps the naive output mapping and launch geometry and adds only explicit data reuse. Each block uses $2T^2 \times 4$ bytes of shared memory. Tiling trades extra synchronization and shared-memory operations for fewer global loads, so its net benefit has to be measured.

6.5 Optimization 2: Tile and Block Size

The same tiled source supports $T = 16$ and $T = 32$, and this experiment changes the tile size and block size together. A larger tile gives more reuse and fewer phases, but it also increases the threads and shared memory per block. Table 14 summarizes the configurations at $N = 4096$. The barrier counts are source-level calls per block, not measured waiting times.

Table 14: Multiplication launch configuration at $N = 4096$.

Version	Threads/block	Blocks	Shared (KiB)	Phases	Barriers/block
Naive	256	65536	0	–	0
Tiled 16	256	65536	2	256	512
Tiled 32	1024	16384	8	128	256

All versions launch 16,777,216 threads and 524,288 warps at this size. A block has 8 warps for the 16-wide tile and 32 warps for the 32-wide tile. These totals say nothing about how many threads are resident at once. We added no manual unroll directive, although the compiler may optimize the fixed-size inner loop, so this experiment compares tile sizes and does not isolate loop unrolling.

6.6 Timing Results

Table 15 gives the median times.

Table 15: Median execution times and tiled-16 speedup.

N	Naive (ms)	Tiled 16 (ms)	Tiled 32 (ms)	Naive/Tiled 16
1000	11.142203	9.168911	11.701505	1.22×
4096	908.577813	620.536289	793.974766	1.46×

Tiled 16 reduced the median time by 17.71% at $N = 1000$ and 31.70% at $N = 4096$, reaching 218.128 and 221.484 GFLOP/s. Tiled 32 was slower than tiled 16 by 27.62% and 27.95%, even though it needs fewer phases and barrier calls. At each size, the three-run timing ranges of the implementations did not overlap.

At $N = 1000$, the tiled loops pad the dimension to 1008 for tiled 16 and 1024 for tiled 32. The GFLOP/s values count only the useful $2N^3$ operations and exclude the padding work. At $N = 4096$, neither tile size needs padding, so padding cannot explain why tiled 32 is slower.

6.7 Numerical Accuracy

The CPU reference accumulates the products of the actual FP32 inputs in FP64. For $N \le 128$, every output is compared with this reference. The full checks passed for the naive and tiled-16 versions at $N = 128$, and tiled 16 also passed at $N = 127$, which exercises the partial-tile handling. For larger sizes, every output is checked for finiteness, and 1024 fixed output positions, including the corners, are compared numerically. All implementations use the same positions. A sampled pass does not establish correctness for every entry of a large matrix.

Each checked position must satisfy

$$|C_i - C_i^{\mathrm{ref}}| \le 10^{-4} + 10^{-4}|C_i^{\mathrm{ref}}|.$$

All checks passed with zero mismatches and zero nonfinite outputs. For the checked set S, the normalized error is

$$E_{\infty,S} = \frac{\max_{i \in S} |C_i - C_i^{\mathrm{ref}}|}{\max_{i \in S} |C_i^{\mathrm{ref}}|}.$$

Table 16 lists the errors, which were identical for all three implementations.

Table 16: Sampled numerical errors, identical across the three versions.

N	Sampled maximum absolute error	$E_{\infty,S}$
1000	$3.76072527 \times 10^{-5}$	$1.68033933 \times 10^{-6}$
4096	$4.74550976 \times 10^{-4}$	$2.01829064 \times 10^{-6}$

Tiling shows no accuracy penalty at the checked positions. This is consistent with each output still being accumulated in the same order over k. The results do not establish bitwise equality of all outputs, and we did not compare GPU FP32 and FP64 performance.

6.8 Profiling Analysis

We used Nsight Compute to profile one kernel launch at $N = 4096$ after 10 warm-up launches. Each profile needed 29 replay passes, and the profiled application ran one timed launch instead of 100. The application times printed under profiling include collection overhead, so they are excluded from all performance comparisons. Table 17 lists the results, with counts in billions where marked.

Table 17: Multiplication profiling at $N = 4096$.

Metric	Naive	Tiled 16	Tiled 32
Registers/thread	40	40	40
Theoretical occupancy (%)	100	100	66.67
Achieved occupancy (%)	99.95	99.93	66.66
Executed warp instructions (10^9)	11.709	8.486	6.608
Global-load L1/TEX sectors (10^9)	8.590	1.074	0.537
L2 read sectors from L1/TEX (10^9)	1.0734	1.0734	0.5369
L1 global-load hit rate (%)	87.50	0.03	0
Useful bytes/global-load sector	18	32	32
L1/TEX throughput (%)	97.37	91.72	60.92
Compute (SM) throughput (%)	64.90	71.22	50.94
Branch efficiency (%)	100	100	100

Why tiled 16 was faster. Compared with the naive kernel, tiled 16 cut the global-load requests by a factor of 16 and the global-load L1/TEX sectors by about a factor of eight. Executed warp instructions fell by 27.5%, while register use and achieved occupancy were essentially unchanged. The naive kernel was also using the L1/TEX path heavily, at 97.37% of its reported throughput. Together, these numbers support less instruction and L1/TEX work as contributors to the speedup. The Compute (SM) throughput was dominated by load/store pipeline activity in all three profiles, so it should not be read as FP32 arithmetic utilization.

The naive kernel already had an 87.5% L1 global-load hit rate, and its L2 read sectors were almost the same as those of tiled 16, at about 1.0734×10^9. The eightfold drop in L1/TEX sectors therefore does not mean an eightfold drop in external memory traffic. Explicit shared-memory reuse replaced much of the reuse that the L1 cache had been providing. DRAM throughput was unavailable, so we make no claim about DRAM bandwidth utilization.

Why tiled 32 was slower. Tiled 32 halved the global-load sectors again and executed 22.1% fewer warp instructions than tiled 16. However, its theoretical occupancy fell to 66.67%, and the measured value was 66.66%. The profiler reported a limit of one resident block per SM. One block has 32 warps, while the hardware allows 48 per SM. The register count per thread did not change, but each block is four times larger. On an SM with 65,536 registers, one 1024-thread block already uses 40,960, so a second block cannot fit.

Fewer resident warps leave less room to hide latency, which fits the lower compute and L1/TEX throughput. Sampled MIO-throttle states rose from 27.4% to 37.2% of all warp samples, consistent with more pressure on the instruction queue that shared-memory operations use. Fewer barrier calls did not remove waiting: the not-issued barrier samples rose from 180,873 to 284,837. These sampled states are diagnostics, not direct measurements of time.

Hardware shared-store bank-conflict counters also rose, from about 19.75 million to 29.27 million, but the source-level excessive shared-wavefront counters were zero in both tiled profiles. These two diagnostics disagree, so we do not assign the slowdown to bank conflicts alone. The clearest measured trade-off is less instruction and global-load work against lower resident parallelism and more shared-memory pressure.

Divergence. All three profiles reported 100% branch efficiency and no divergent branches, so the performance differences are not explained by branch divergence. This applies to the profiled, tile-divisible size $N = 4096$.

6.9 Summary

Tiling with 16×16 blocks reduced execution time by 17.71% at $N = 1000$ and 31.70% at $N = 4096$. Increasing the tile to 32×32 reduced global-load work and the number of synchronization phases, but it lowered occupancy and made the kernel about 28% slower than tiled 16. We observed no accuracy change at the checked positions.

We keep tiled 16 for the tested workloads. The main lesson is that data reuse must be balanced against block resource needs, resident parallelism, and synchronization cost. Fewer loads, instructions, or barrier calls do not by themselves guarantee faster execution. These experiments did not establish the largest supported matrix size or GPU FP64 performance.

7 Comparative Analysis

The experiments show that the value of an optimization depends on how a kernel uses its data, how much parallel work it has, and what resources it needs. We do not rank the four operations by raw execution time, because they do very different amounts of work: trace reduces N diagonal elements, transpose and addition process N^2 elements, and multiplication performs $O(N^3)$ arithmetic. All comparisons below are between versions of the same operation. The references supply algorithmic background and help interpret the metrics, while all measurements come from our experiments on the Orin.

7.1 Data Access Determines the Value of an Optimization

Shared memory did different jobs in different kernels. Transpose used it to reorganize memory accesses, while multiplication used it to share input values among several output dot products. This follows the CUDA memory model and the standard tiling patterns [4, 5]. In transpose, tiling reduced store-sector traffic without changing the number of output elements. In multiplication,

tiled 16 reduced instruction and L1/TEX work, but its L2 read sectors stayed close to those of the naive kernel, whose reuse was already handled by the cache. A reduction at one level of the memory hierarchy should therefore not be described as an equal reduction in DRAM traffic.

Addition has no input reuse, since each input value feeds one output and the baseline already read adjacent elements. Its gain came from changing the work per thread while keeping that access pattern. Trace kept its strided diagonal loads even after the reduction was reorganized. In each case, the useful optimization was the one that matched how the kernel actually uses its data.

7.2 Available Work and Resident Parallelism

Launching fewer blocks helped in some kernels and hurt in others. Trace pair-load halved the first-stage blocks and reduced the executed work, but achieved occupancy fell from 28.89% to 15.59% and the timing did not clearly improve. Addition cut its block count by about four, yet achieved occupancy rose from 86.90% to 96.01% and the time improved, because the grid still held many waves of work.

Multiplication showed a different limit. Tiled 32 kept 40 registers per thread but raised the block size to 1024 threads. The profiler reported one resident block per SM and about 66.7% occupancy, compared with nearly 100% for tiled 16. The kernel ran about 28% slower despite executing fewer instructions and loading fewer global sectors. Pair-load in trace therefore ran short of available work, while tiled 32 ran short of resident parallelism because of its resource needs. Occupancy and launch counts have to be read together and not optimized as separate targets [2, 3].

7.3 Synchronization and Implementation Complexity

Adding barriers improved transpose and multiplication, because the better access organization or data reuse outweighed their cost. However, halving the number of tile phases and barrier calls in multiplication did not make tiled 32 faster, so barrier count alone cannot predict synchronization cost or total time.

Implementation complexity also matters. Padding in transpose cost only 64 extra bytes of shared memory per block and cut time by a further 6.8–7.4% beyond tiling. Addition's four-element mapping gave a modest 3.93–4.92% gain with little extra code. Trace pair-load showed no clear benefit, so we documented it but did not keep it. Our selection rule was to prefer a clear implementation with a repeatable timing benefit and passing correctness checks. Table 18 lists the implementations we retained.

Table 18: Retained implementations for the tested workloads.

Operation	Implementation	Block configuration	Main change
Transpose	Padded shared-memory tile	16×16	Access reordering and padding
Trace	Sequential reduction	128 threads	Reduction organization
Addition	Four elements per thread with loop expansion	128 threads	Work per thread
Multiplication	Shared-memory tiled kernel	16×16	Explicit input reuse

7.4 Accuracy and Scope of the Evidence

Floating-point rounding can depend on the order of operations, especially in reductions [6]. In trace, changing the reduction tree changed the errors slightly. Addition and multiplication gave the same error metrics for all the versions we evaluated, and all checks passed. For large multiplication matrices, we checked every output for finiteness but compared numerically at only 1024 fixed positions, so that validation is sampled and not exhaustive.

We assessed FP32 results against reference values. We did not test the performance cost of switching the GPU to FP64, determine the largest supported matrix size, or measure power and energy. A shorter execution time does not by itself mean lower energy use, so we draw no energy conclusions.

8 Conclusion

This project implemented and benchmarked four CUDA matrix operations on an NVIDIA Orin. We used ordinary timing runs to measure performance and profiling to examine execution behavior. We retained padded transpose, sequential trace reduction, four-element addition with loop expansion, and multiplication with 16×16 shared-memory tiles.

The results show that a useful optimization has to organize data access, reduce instruction work, and keep enough parallel work running. At $N = 4096$, padded transpose was $1.85\times$ faster than the naive kernel, and tiled-16 multiplication cut time by 31.70%. Sequential trace reduced time by 17.1% in the fixed-256-thread comparison, and the separate block-size study favored 128 threads. Four-element addition cut time by 4.92% in the fixed-128-thread comparison. The two experiments that did not pay off, pair-load trace and 32×32 multiplication tiles, show that fewer blocks, instructions, or barrier calls do not guarantee a speedup.

Our final choices balance the measured benefit against implementation complexity and the correctness evidence we have. Future work could test larger matrices and other GPU precision settings, and could measure energy use with dedicated power measurements. These conclusions apply to the tested workloads and configurations.

References

[1] M. Harris, *Optimizing Parallel Reduction in CUDA*. NVIDIA Developer Technology, presentation slides. Online PDF. Accessed Oct. 19, 2026.

[2] NVIDIA, *CUDA C++ Best Practices Guide*, CUDA Toolkit 12.6. Online documentation. Accessed Oct. 19, 2026.

[3] NVIDIA, *Nsight Compute Profiling Guide*. Online documentation. Accessed Oct. 19, 2026.

[4] NVIDIA, *CUDA C++ Programming Guide*, CUDA Toolkit 12.6. Online documentation. Accessed Oct. 19, 2026.

[5] M. Harris, "An Efficient Matrix Transpose in CUDA C/C++," *NVIDIA Technical Blog*, Feb. 18, 2013. Online article. Accessed Oct. 19, 2026.

[6] NVIDIA, *Floating Point and IEEE 754 Compliance for NVIDIA GPUs*, CUDA Toolkit 12.6 documentation. Online documentation. Accessed Oct. 19, 2026.